

ComicCraft – AI Comic Story Creator using Gemini Models

Phase 4 – Project Planning

Phase 4 – Project Planning

Project Planning defines how ComicCraft will be developed, organized, tested, and prepared for implementation. The phase converts the requirements and design into a practical development plan.

1. Objectives

- Plan the development activities for ComicCraft.
- Divide the application into frontend, backend, and AI components.
- Select the software tools and technologies required for implementation.
- Define the project folder structure and development workflow.
- Plan API integration with the Gemini model.
- Identify possible project risks and suitable solutions.
- Prepare the project for coding, testing, and demonstration.

2. Planned System Components

Component	Technology	Purpose
User Interface	HTML, CSS, JavaScript	Accept story prompts and display generated comic content
Backend	Python, FastAPI	Receive requests and communicate with the AI model
AI Engine	Gemini Models	Generate titles, characters, scenes, dialogues, and panel descriptions
API Communication	HTTP/JSON	Transfer prompts and generated results between frontend and backend
Configuration	.env	Store the Gemini API key securely
Version Control	Git/GitHub	Maintain and manage project source code

3. Development Plan

Step	Activity	Planned Work
Step 1	Project Setup	Create the project folders and install required Python packages.
Step 2	Frontend	Create the prompt input page, button, and result display area.
Step 3	Backend	Create the FastAPI application and /generate API endpoint.
Step 4	Gemini Integration	Connect the backend to the Gemini API and prepare the

		comic-generation prompt.
Step 5	API Connection	Connect JavaScript to the FastAPI endpoint using fetch().
Step 6	Output Display	Display the generated title, characters, scenes, dialogues, and panels.
Step 7	Testing	Test valid prompts, empty prompts, API errors, and generated responses.
Step 8	Final Preparation	Clean the interface, verify configuration, and prepare the project for demonstration.

4. Planned Project Structure

```
ComicCraft/
|
|— main.py
|— requirements.txt
|— .env
|
|— templates/
|   |— index.html
|
|— static/
|   |— style.css
|   |— script.js
```

5. Environment and Installation Coding

Install the packages required for the ComicCraft backend:

```
pip install fastapi uvicorn google-genai python-dotenv
```

The requirements.txt file will contain:

```
fastapi
uvicorn
google-genai
python-dotenv
```

The Gemini API key will be stored in the environment file:

```
GEMINI_API_KEY=your_api_key
```

The .env file should not be uploaded to a public repository. A .gitignore file can be used to prevent the API key from being committed.

```
.env
__pycache__/
*.pyc
```

6. Backend Planning Code

The planned backend receives a prompt through a POST request and sends it to Gemini. The API key is loaded from the environment.

```
from fastapi import FastAPI
from pydantic import BaseModel
```

```

from google import genai
from dotenv import load_dotenv
import os

load_dotenv()

app = FastAPI()

client = genai.Client(
    api_key=os.getenv("GEMINI_API_KEY")
)

class ComicRequest(BaseModel):
    prompt: str

@app.post("/generate")
def generate_comic(request: ComicRequest):
    instruction = f"""
    Create a simple comic story from this idea:

    {request.prompt}

    Include:
    1. Title
    2. Characters
    3. Scenes
    4. Dialogues
    5. Comic panel descriptions
    """

    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=instruction
    )

    return {"comic": response.text}

```

7. Frontend Planning Code

The frontend will provide a simple input form and use JavaScript to send the prompt to the backend.

```

<textarea id="prompt"
placeholder="Enter your comic story idea..."></textarea>

<button onclick="generateComic()">
    Generate Comic
</button>

<div id="result"></div>

```

JavaScript will connect the interface to the backend:

```

async function generateComic() {
    const prompt = document.getElementById("prompt").value;

    const response = await fetch("/generate", {
        method: "POST",
        headers: {
            "Content-Type": "application/json"

```

```

    },
    body: JSON.stringify({ prompt: prompt })
  });

  const data = await response.json();

  document.getElementById("result").innerText =
    data.comic;
}

```

8. Development Timeline

Time	Planned Activity
Week 1	Project setup and requirement review
Week 2	Frontend interface development
Week 3	FastAPI backend development
Week 4	Gemini API integration
Week 5	Frontend-backend integration
Week 6	Testing and bug fixing
Week 7	Final improvements and documentation
Week 8	Project demonstration preparation

9. Risk Management

Risk	Possible Effect	Planned Solution
Invalid or missing API key	Gemini request fails	Store the key in .env and verify configuration before running.
Network/API failure	Comic generation may not complete	Show an error message and retry the request when appropriate.
Empty user prompt	Poor or meaningless output	Validate that the prompt is not empty before sending it.
Unexpected AI response	Output may not follow the expected structure	Use a clear generation instruction and validate the response.
API key exposure	Security risk	Never place the API key directly in frontend code or public repositories.

10. Planned Testing Checklist

- Check whether the home page loads correctly.
- Check whether a valid story prompt is accepted.
- Check whether an empty prompt is rejected or handled properly.
- Check whether the /generate endpoint receives JSON correctly.
- Check whether Gemini returns generated content.
- Check whether the generated result appears on the webpage.
- Check error handling when the API key or network is unavailable.
- Check the interface on different screen sizes.

11. Expected Result

At the end of the planning phase, the development team will have a clear implementation plan, folder structure, technology list, development schedule, API integration approach, risk-management plan, and testing checklist. These items provide the foundation for the next development stage.

12. Phase 4 Summary

Phase 4 – Project Planning organizes the complete development process of ComicCraft. The project is divided into frontend, backend, and AI components. The planned coding workflow uses HTML, CSS, JavaScript, Python FastAPI, and Gemini Models. The phase also defines the development timeline, project risks, environment setup, and testing plan.